

JUNE 2026

**GAZİ UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING**

Burak ÇETİN – 22118080032

Çağrı ÇELİK – 22118080069

Submission 2 – Implementation Report

**Accelerating 2D Image Convolution on GPU: A Performance
Study of Memory-Hierarchy-Aware CUDA Implementations**

**CENG479 – PARALLEL COMPUTER ARCHITECTURES AND
PROGRAMMING**

Source code:

<https://github.com/brakcetin/cuda-2d-convolution-optimization>

CONTENTS

1. Introduction	3
2. Sequential Baseline Implementation	3
3. Parallel Implementation	4
4. Performance Comparison	5
5. Academic Background	12
6. Challenges and Solutions	12
7. Conclusion and Future Improvements	13
8. References	13

1. Introduction

Two-dimensional convolution is a core operation in image processing and computer vision. Gaussian blur, sharpening, Sobel edge detection, denoising, and the convolutional layers of neural networks all apply the same pattern: every output pixel is a weighted sum of an input neighborhood. For an image of size $W \times H$ and a square filter of size $k \times k$, direct convolution costs $W * H * k^2$ multiply-accumulate operations. A 4096x4096 image with an 11x11 filter needs more than two billion operations, and a sequential CPU implementation of that case takes about three seconds on our development laptop. Submission 1 proposed to accelerate this operation with CUDA and to measure how each level of the GPU memory hierarchy contributes to the speedup.

This report presents the completed implementation and its measured results. We implemented one sequential CPU baseline and six CUDA versions of the same grayscale convolution: a naive global-memory kernel, a shared-memory tiled kernel, a shared-memory kernel with constant-memory filter coefficients, a multi-output kernel, a register-tiled kernel, and a separable two-pass kernel for filters that decompose into 1D passes. Every CUDA result in this report passed correctness verification against the CPU output before being included in the analysis.

The official benchmark sweeps four image sizes (512x512 to 4096x4096), four filter sizes (3x3 to 11x11), four filter types (box, Gaussian-like, sharpen, Sobel-like), and four CUDA block shapes (8x8, 16x16, 32x8, 32x16), with five timed repeats and one warm-up per case. This produces 1408 timing rows per GPU. We ran the full matrix on two laptops: a GTX 1650 Max-Q, which is the official baseline hardware of the project, and an RTX 4070 Laptop GPU as secondary hardware evidence. On the GTX 1650, the best kernel-only speedup over the CPU baseline is 744.2x and the best end-to-end speedup including transfers is 56.2x. On the RTX 4070, the same code reaches a peak of 2383 GFLOP/s, 3.9x the GTX 1650 peak, without any source change.

The source code of the sequential baseline, all CUDA versions, the benchmark runner, and the plotting scripts is available at <https://github.com/brackctin/cuda-2d-convolution-optimization>. The rest of the report follows the required structure: Section 2 describes the sequential baseline, Section 3 the CUDA kernel designs, Section 4 the performance comparison with tables and graphs, Section 5 the academic background, Section 6 the challenges we hit and how we solved them, and Section 7 the conclusions and future improvements.

2. Sequential Baseline Implementation

The sequential baseline (`src/convolution_cpu.cpp`) is a single-threaded C++17 implementation. It loops over output rows, output columns, filter rows, and filter columns. Out-of-image coordinates use zero-padding semantics: positions outside the image contribute zero to the sum. The image is stored as a flat `std::vector<float>` in row-major order, and the code is compiled with MSVC in Release mode with standard optimizations.

The baseline serves two roles. First, it is the reference implementation: every CUDA output is compared against the CPU output of the same case, element by element, with maximum and mean absolute error reported per run. Second, it provides the denominator for all speedup figures, measured with `std::chrono::steady_clock` over the same repeat protocol as the GPU versions.

We kept the baseline single-threaded on purpose. The course project compares sequential execution against GPU parallelism, so adding OpenMP or CPU SIMD to the baseline would mix two parallelization studies into one number. The baseline also implements a separable CPU path (two 1D passes) which is used only as the correctness reference for the separable CUDA version, since comparing a two-pass GPU result against a direct 2D CPU result would conflate algorithmic and numerical differences.

The cost model of the baseline matches the textbook formula. Measured CPU time grows close to linearly with $W * H * k^2$: on the GTX 1650 machine, 512x512 with a 3x3 box filter takes 5.9 ms while 4096x4096 with an 11x11 Gaussian-like filter takes 3170.4 ms, a factor of about 537 for a workload that is 538 times larger.

3. Parallel Implementation

All CUDA versions compute the same zero-padded convolution and share one thread-mapping idea: output pixels are independent, so each thread (or small group of outputs per thread) can work without synchronization across blocks. The launch grid is $\text{ceil}(W / \text{block_width}) \times \text{ceil}(H / \text{block_height})$ blocks. Block shape is a runtime parameter, validated so that $\text{block_width} * \text{block_height} \leq 1024$. Table 3.1 summarizes the six kernels and which memory-hierarchy idea each one tests.

Table 3.1. Implemented CUDA versions and the idea each one tests.

Version	One thread computes	Image reads	Filter reads	Idea tested
cuda_naive_global_memory	1 pixel	global	global	parallelism alone
cuda_shared_memory_tiled	1 pixel	shared tile + halo	global	input reuse in shared memory
cuda_shared_constant_filter	1 pixel	shared tile + halo	constant	shared input + broadcast filter
cuda_multi_output	2 horizontal pixels	global	global	fewer threads, more work per thread
cuda_register_tiled	2 horizontal pixels	global	global	register accumulators, read reuse
cuda_separable	1 pixel per pass	global	global	work reduction k^2 to $2k$

Naive global-memory kernel. One thread computes one output pixel and reads every input pixel and filter coefficient from global memory. Neighboring threads re-read overlapping neighborhoods, so the kernel is memory-bound. It is the parallel baseline that isolates what pixel-level parallelism alone achieves.

Shared-memory tiled kernel. Each block first cooperatively loads its input tile plus a halo of radius $= k / 2$ pixels on each side into dynamic shared memory, synchronizes with `__syncthreads()`, and then computes from the shared tile. The shared allocation is $(\text{block_width} + 2 * \text{radius}) * (\text{block_height} + 2 * \text{radius})$ floats, which ties shared-memory footprint to the launch shape and makes block geometry a performance variable.

Shared + constant filter kernel. Same input tiling, but filter coefficients live in `__constant__` memory, copied once per case with `cudaMemcpyToSymbol`. All threads of a warp read the same coefficient at the same step, which is the broadcast access pattern constant memory is built for. The filter limit is 31x31 coefficients, far above the largest benchmarked filter.

Multi-output and register-tiled kernels. Both assign two horizontally adjacent outputs to one thread, halving the launched thread count. The register-tiled variant keeps both accumulators in registers and walks the filter window so overlapping input reads serve both outputs. Neither kernel reduces the mathematical operation count; they test whether thread-level work assignment and register reuse beat one-thread-one-pixel scheduling.

Separable kernel. Box and Gaussian-like filters are generated from 1D vectors, so a $k \times k$ convolution factors into a horizontal k -tap pass followed by a vertical k -tap pass. Arithmetic per pixel drops from k^2 to $2k$ multiply-accumulates; for 11×11 that is 121 versus 22. The cost is one intermediate image written to and read back from global memory between passes. Sharpen and Sobel-like filters are treated as non-separable in this project, so this version is reported only for box and Gaussian-like filters.

Timing and correctness instrumentation. CPU time uses `std::chrono`; kernel time uses CUDA events around the kernel launch only; total GPU time adds allocation, host-to-device copy, device-to-host copy, and free time. Each case runs one warm-up plus five timed repeats and the CSV records average, minimum, maximum, and standard deviation, estimated operation counts, and GFLOP/s. Every CUDA run is compared against the CPU reference with a pass threshold of $1e-4$ maximum absolute error.

4. Performance Comparison

4.1 Experimental setup. Table 4.1 lists the benchmark matrix.

Table 4.1. Official benchmark matrix (identical on both GPUs).

Parameter	Values
Image sizes	512x512, 1024x1024, 2048x2048, 4096x4096
Filter sizes	3x3, 5x5, 7x7, 11x11
Filter types	box, Gaussian-like, sharpen, Sobel-like
Block shapes	8x8, 16x16, 32x8, 32x16
Repeats / warm-ups	5 / 1
Rows per GPU	1408
Official GPU	NVIDIA GeForce GTX 1650 with Max-Q Design
Secondary GPU	NVIDIA GeForce RTX 4070 Laptop GPU

Synthetic grayscale images with uniform random values in $[0, 1]$ keep every run reproducible and keep image decoding out of the timing path. All 1408 rows passed correctness on both GPUs; the largest absolute error in either CSV is about $1e-6$, two orders of magnitude below the $1e-4$ tolerance. The two GPUs also produced bit-identical PGM demo outputs for the qualitative real-image path.

We report two speedup numbers per case. Kernel-only speedup (CPU time / CUDA kernel time) measures the computation itself once data is on the GPU. Total speedup (CPU time / full allocate-copy-compute-copy-free time) is what an application sees when the image starts and ends in host memory. The gap between the two is the data-movement cost, and it is large: transfers and allocation dominate total time in every case we measured.

4.2 Official GTX 1650 results. Table 4.2 lists the headline numbers.

Table 4.2. Headline results, GTX 1650 Max-Q, official matrix.

Metric	Result
Best kernel-only speedup	744.216x, cuda_separable, 4096x4096, 11x11 Gaussian-like, 32x8 block
Best total speedup	56.205x, same case
Best direct-convolution kernel speedup	432.778x, cuda_shared_constant_filter, 4096x4096, 11x11 Sobel-like, 32x16
Best output-tiling kernel speedup	409.608x, cuda_register_tiled, 512x512, 3x3 Sobel-like, 16x16
Peak kernel throughput	617 GFLOP/s, cuda_shared_constant_filter, 1024x1024, 11x11 Sobel-like

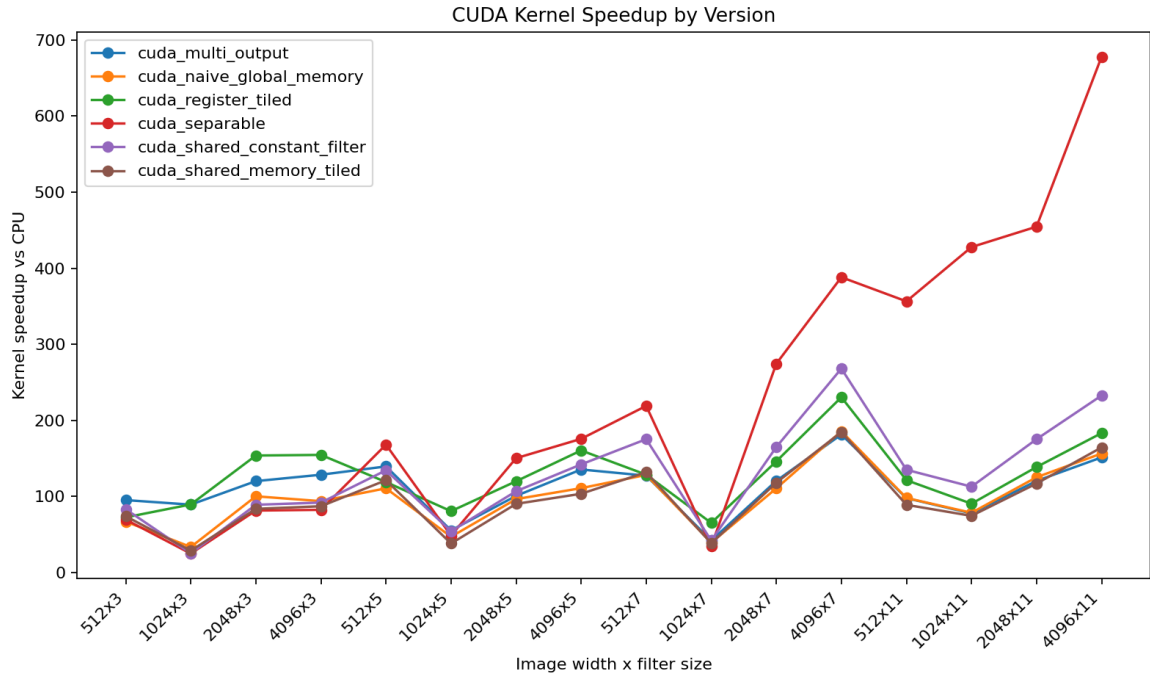


Figure 4.1. Kernel-only speedup by CUDA version across the full GTX 1650 matrix (best block shape per point).

Table 4.3. Representative cases, GTX 1650 Max-Q.

Image	Filter	Type	CPU avg (ms)	Best kernel: version / block / speedup	Best total: version / block / speedup
512x512	3x3	box	5.864	cuda_register_tiled / 32x8 / 117.800x	cuda_shared_memory_tiled / 8x8 / 2.831x
1024x1024	7x7	sobel	60.089	cuda_shared_constant_filter / 32x16 / 146.015x	same / 13.634x
2048x2048	11x11	sobel	609.550	cuda_shared_constant_filter / 32x16 / 355.709x	same / 47.434x
4096x4096	11x11	gaussian	3170.441	cuda_separable / 32x8 / 744.216x	same / 56.205x
4096x4096	11x11	sobel	3063.866	cuda_shared_constant_filter / 32x16 / 432.778x	cuda_register_tiled / 32x8 / 46.974x

Speedup grows with workload size. At 512x512 the kernels finish in tens of microseconds and fixed overheads cap the total speedup near 3x. At 4096x4096 the same kernels deliver total speedups near 50x and kernel speedups in the hundreds. Figures 4.2 and 4.3 show both trends; Figure 4.4 shows how transfer and allocation costs dominate total time even when the kernel is fast.

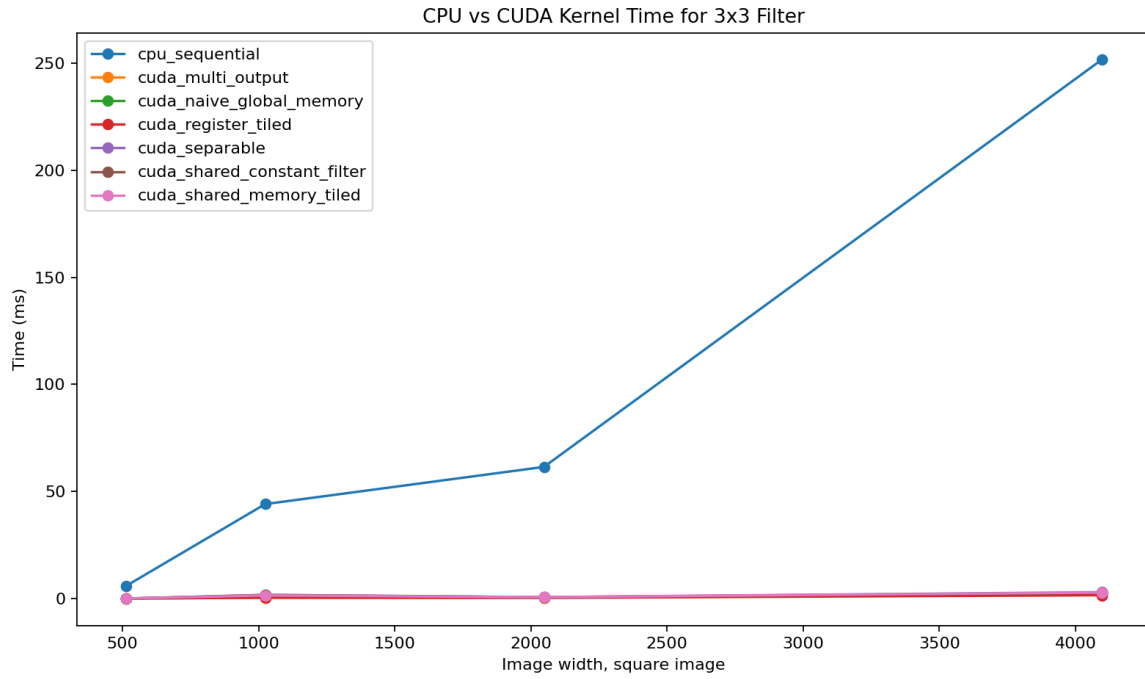


Figure 4.2. CPU and CUDA kernel time versus image size, 3x3 filters, GTX 1650.

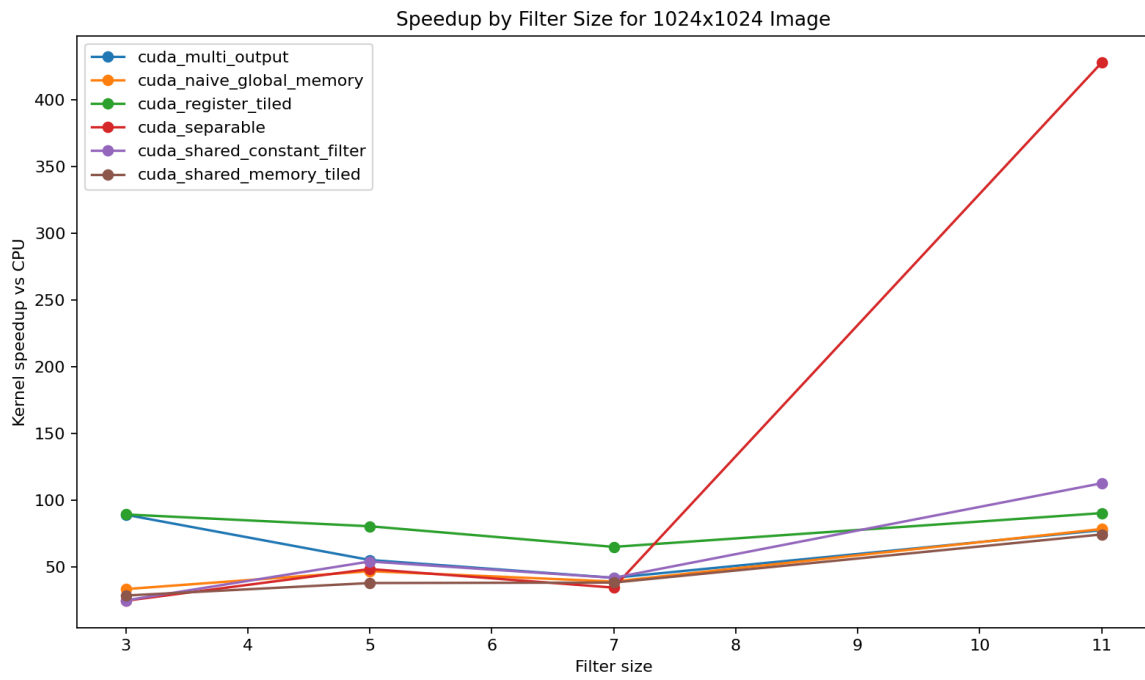


Figure 4.3. Kernel speedup versus filter size at 1024x1024, GTX 1650.

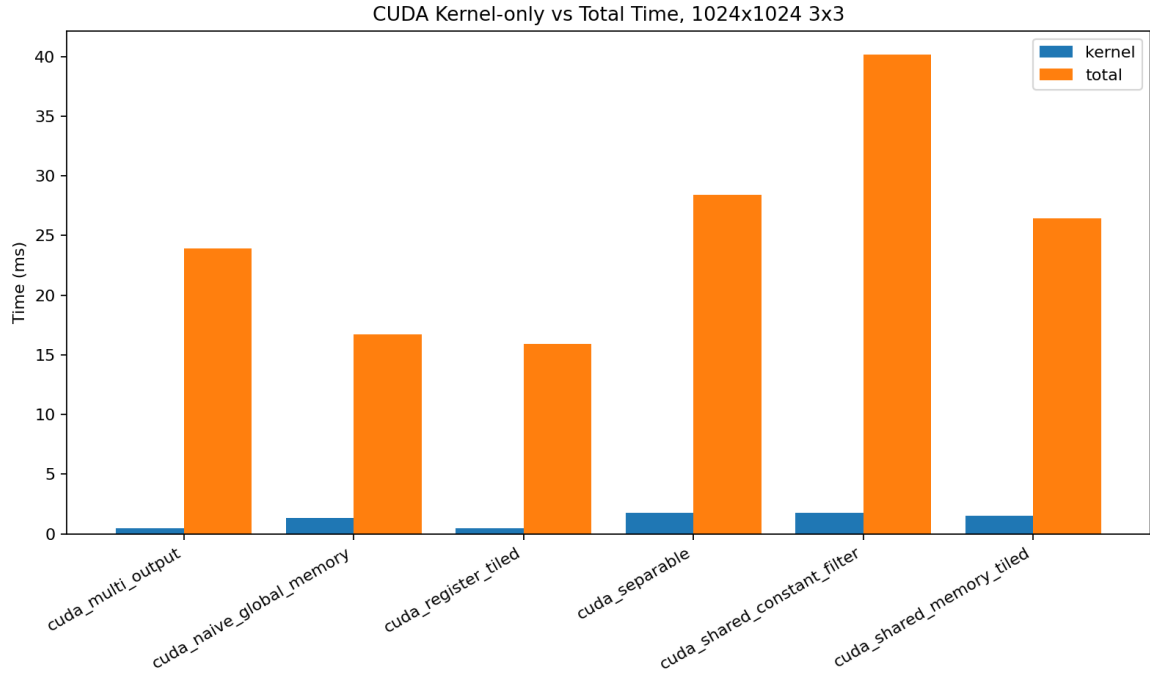


Figure 4.4. Kernel-only versus total GPU time at 1024x1024 with 3x3 filters, GTX 1650. The orange bars are dominated by allocation and host-device transfers.

The per-version breakdown at 4096x4096 with the 11x11 Sobel-like filter (Table 4.4, Figure 4.6) isolates what each optimization adds. Two observations stand out. First, shared-memory tiling alone does not beat the naive kernel here; the GTX 1650 L1/L2 caches already capture much of the neighborhood reuse, and halo loading adds overhead. Second, moving the filter coefficients to constant memory halves the kernel time. With an 11x11 filter every thread reads 121 coefficients per output pixel, so the broadcast-friendly constant cache removes a large share of memory traffic. The two output-tiling kernels land between these results: register tiling beats one-pixel-per-thread scheduling in many smaller cases (it holds the best 512x512 result at 409.6x) but does not reach the constant-filter kernel on large direct filters.

Table 4.4. Direct-convolution versions at 4096x4096, 11x11 Sobel-like, GTX 1650 (best block shape per version).

Version	Best kernel time (ms)	Kernel speedup	Total speedup
cuda_naive_global_memory	13.584	225.6x	30.6x
cuda_shared_memory_tiled	13.768	222.5x	39.2x
cuda_multi_output	19.789	154.8x	34.7x
cuda_register_tiled	16.318	187.8x	38.3x
cuda_shared_constant_filter	7.080	432.8x	36.6x

For the same image with the separable 11x11 Gaussian-like filter, cuda_separable needs 4.260 ms where the best direct kernel needs 7.396 ms. Two 11-tap passes replace one 121-tap pass, so the win is algorithmic rather than a memory-hierarchy effect, and it holds the overall record at 744.2x kernel speedup.

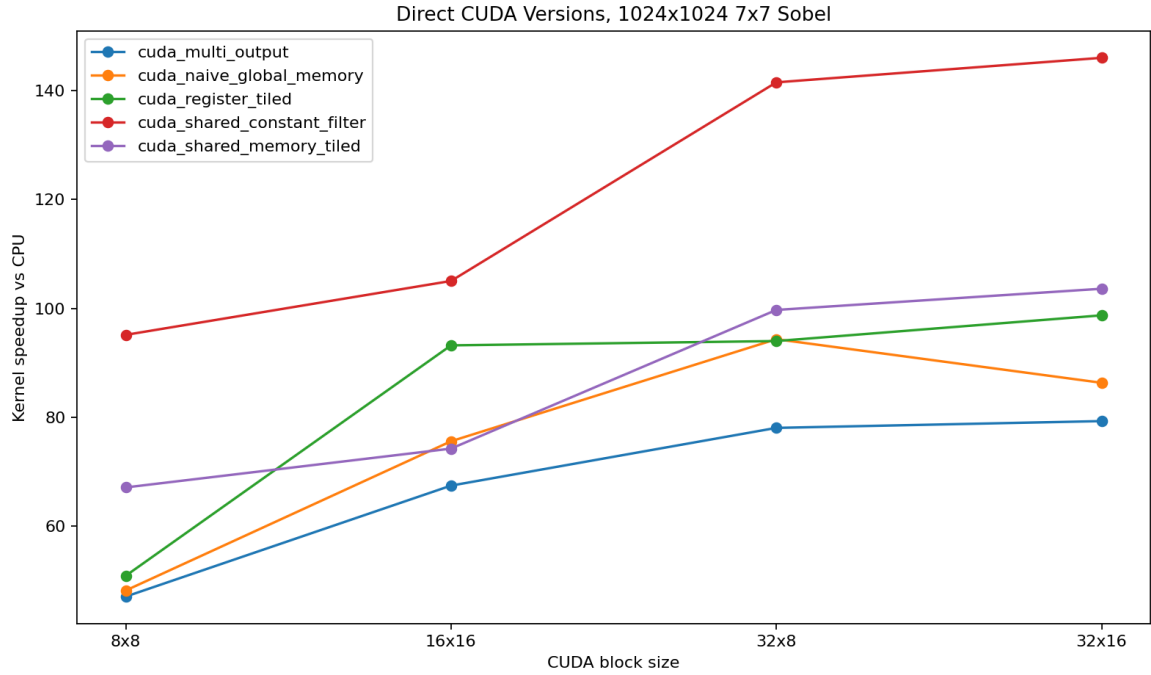


Figure 4.5. Direct CUDA versions at 1024x1024, 7x7 Sobel-like, GTX 1650.

Block shape matters and no single shape wins everywhere. Across the 64 image/filter/type cases on the GTX 1650, the kernel-time winner used 32x8 in 42 cases, 32x16 in 12, 16x16 in 9, and 8x8 in 1. Wide blocks favor coalesced row-major reads. The benchmark therefore records the winning shape per case instead of fixing one launch configuration (Figure 4.6).

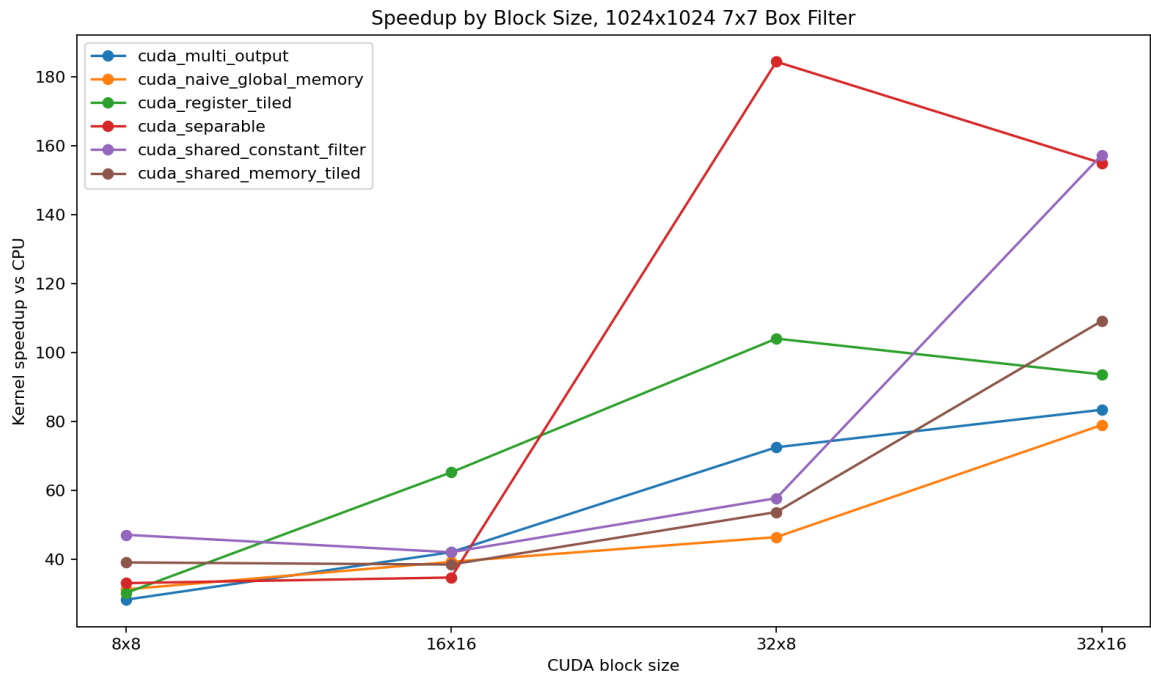


Figure 4.6. Effect of block shape at 1024x1024, 7x7 box filter, GTX 1650.

Figures 4.1 to 4.6 are generated from the official CSV by scripts/plot_results.py and are committed under results/plots/.

4.3 Secondary hardware: RTX 4070 Laptop GPU. The full 1408-row matrix was rerun without source changes on the second team laptop. Speedup factors are not comparable across the two machines because the host CPUs differ as well: the RTX machine's CPU finishes the 4096x4096 11x11 case in 1334 ms versus 3170 ms on the GTX machine. Raw kernel times and GFLOP/s are comparable, and Table 4.5 and Figure 4.7 use those.

Table 4.5. Hardware comparison on the same benchmark matrix. Speedups are relative to each machine's own CPU.

Metric	GTX 1650 Max-Q	RTX 4070 Laptop	Ratio
Best kernel time, 1024x1024 7x7 Sobel-like (ms)	0.412	0.076	5.4x
Best kernel time, 4096x4096 11x11 Sobel-like (ms)	7.080	1.704	4.2x
Best kernel time, 4096x4096 11x11 Gaussian-like (ms)	4.260	1.145	3.7x
Peak kernel GFLOP/s	617	2383	3.9x
Best total speedup vs own CPU	56.2x	79.3x	-

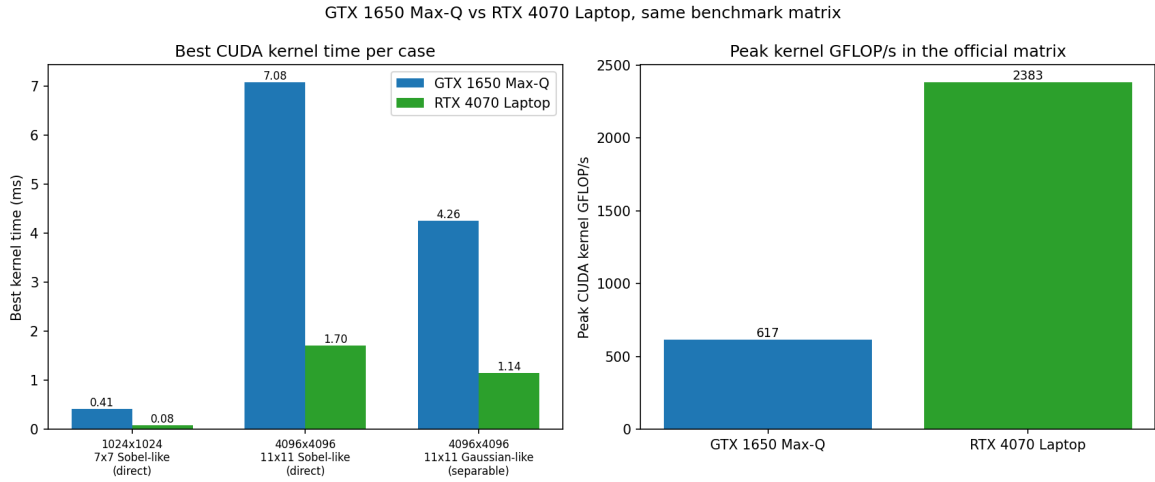


Figure 4.7. GTX 1650 Max-Q versus RTX 4070 Laptop: best kernel times and peak GFLOP/s (scripts/plot_gpu_comparison.py).

The version ranking is stable across hardware: `cuda_shared_constant_filter` is the best direct kernel and `cuda_separable` the best separable kernel on both GPUs. Within its own machine, the RTX 4070 reaches a best kernel-only speedup of 1338.1x (`cuda_separable`, 1024x1024, 11x11 box) and a best total speedup of 79.3x.

Two differences are worth noting. First, on the RTX 4070 the best total-speedup case is the constant-filter Sobel kernel rather than the separable Gaussian kernel: kernels are now so fast (1.1 to 1.7 ms at 4096x4096) that transfer time dominates the total on every version, which compresses end-to-end differences between kernels. Second, the block-shape winner distribution spreads out (16x16 wins 24 of 64 cases, 32x8 wins 20, 32x16 wins 14, 8x8 wins 6), confirming that the best launch configuration depends on hardware as well as workload. Figure 4.8 shows the RTX 4070 version overview; the RTX plots are committed under `results/plots_rtx4070/`.

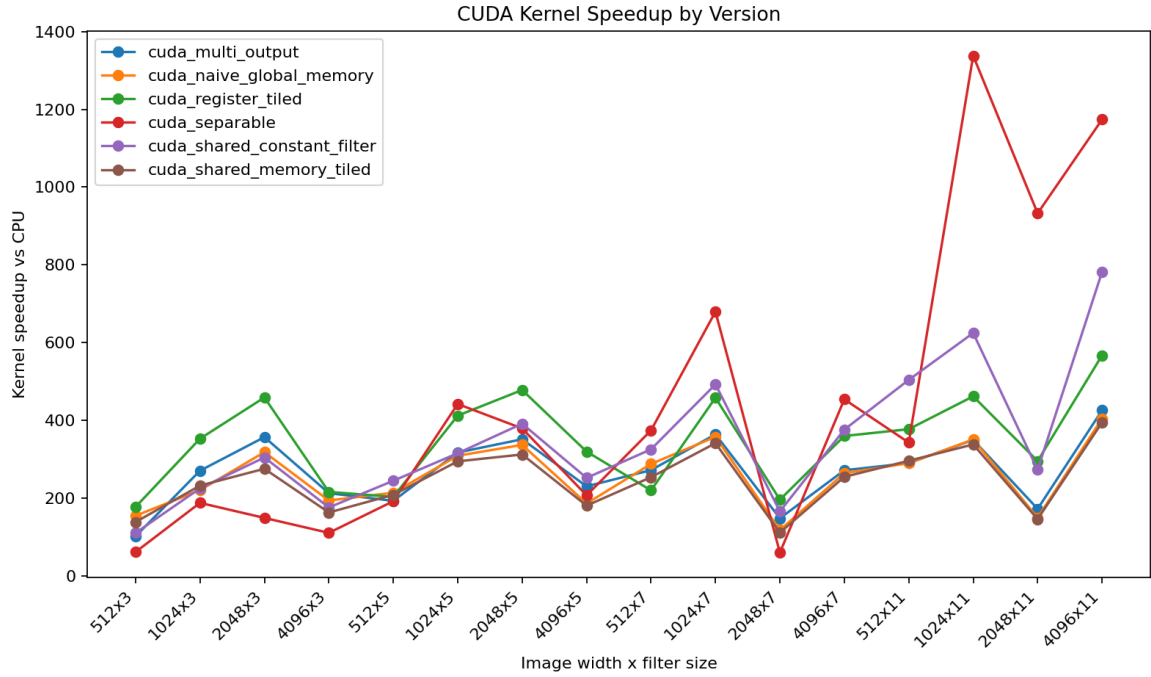


Figure 4.8. Kernel-only speedup by CUDA version across the full RTX 4070 matrix, relative to the RTX machine's own CPU.

4.4 Qualitative real-image demo. Beyond the synthetic matrix, the executable supports a dependency-free grayscale PGM path used for demonstration: Sobel-like edge detection on a building photo and a wood texture, Gaussian-like blur and sharpening on a portrait (Figure 4.9). These outputs are committed under results/ and reproduce bit-identically on both GPUs. They are presentation material; all speedup claims come from the synthetic matrix.

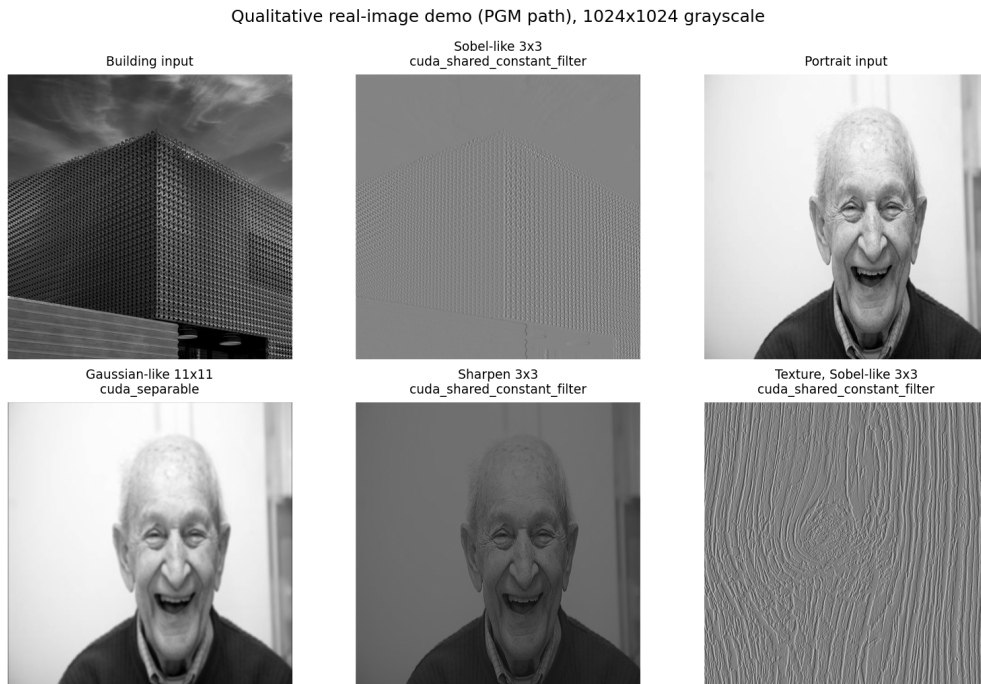


Figure 4.9. Real-image demo outputs (1024x1024 PGM path, scripts/plot_real_image_panel.py). The Sobel-like output maps zero gradient to mid-gray.

5. Academic Background

GPU convolution is usually memory-bound rather than compute-bound, and the literature we build on approaches that problem from different levels of the memory hierarchy. Van Werkhoven, Maassen, Bal, and Seinstra (2014) showed that tiling strategy and thread-block geometry must adapt to filter size and hardware, and that no fixed configuration is optimal across workloads. Our block-shape sweep reproduces that conclusion on laptop GPUs: 32x8 dominates on the GTX 1650 while 16x16 wins a plurality on the RTX 4070, and the winner shifts with image and filter size.

Iandola, Sheffield, Anderson, Phothilimthana, and Keutzer (2013) minimize communication by keeping convolution data in registers, reporting large gains for small filters. Our register-tiled kernel applies the same idea in a simplified 2x1 form, and consistent with their results it performs best in the small-filter regime (best 512x512 3x3 kernel on the GTX 1650) while constant-memory filtering wins for large direct filters. Perrot, Domas, and Couturier (2016) present an optimized direct GPU convolution and document the importance of coalesced access patterns, which matches our observation that wide 32xN blocks tend to win kernel time. Lu, Zhang, and Wang (2020) optimize convolution at the level of GPU memory transactions, the same bottleneck our shared+constant kernel attacks by replacing per-thread global filter reads with broadcast constant-cache reads.

At the library level, Jorda, Valero-Lara, and Pena (2019) evaluate cuDNN's convolution algorithms across filter sizes on Volta GPUs and show that the best algorithm changes with the workload, which mirrors in production form our finding that separable, constant-filter, and register-tiled kernels each win a different region of the parameter space. Allusse, Horain, Agarwal, and Saipriyadarshan (2008) make the application argument with GpuCV: convolution is one stage of larger vision pipelines, and transfer overhead determines how much of the kernel-level gain survives at application level. Our kernel-versus-total gap (744x versus 56x in the headline case) quantifies the same effect.

6. Challenges and Solutions

Boundary agreement between CPU and GPU. Convolution edges are where implementations quietly diverge. We fixed zero-padding semantics across all seven implementations and verified every benchmark case against the CPU reference, so a boundary mistake in any kernel fails its share of the 1408-row matrix instead of hiding.

Shared-memory halo indexing. The tiled kernels load $(\text{block_width} + 2r) \times (\text{block_height} + 2r)$ floats per block, and mixing global and tile-local coordinate systems is error-prone. Testing across four filter sizes and four block shapes caught tile-boundary mistakes early because an indexing error shows up immediately as a correctness failure in some block-shape and filter-size combination.

Honest timing. A single timed run misleads. The benchmark uses one warm-up plus five repeats and records average, minimum, maximum, and standard deviation per case; the standard-deviation columns later exposed real instability (one 2048x2048 shared-tiled case had visibly higher kernel variance than its siblings). Separating kernel-only from total time, with the allocation/copy/free breakdown stored per row, avoided over-claiming end-to-end speedup by two orders of magnitude.

Comparing two machines fairly. The two team laptops differ in both GPU and CPU, so cross-machine speedup ratios are meaningless. We keep each GPU's results in separate CSV files,

never mix them, and compare hardware only through raw kernel times and GFLOP/s (Section 4.3).

Reproducible demo inputs. The PGM conversion script originally fell back to Python/Pillow when ImageMagick was missing, and Pillow's grayscale conversion and resampling produce different pixel values than ImageMagick's. The diff against the committed reference inputs made the run non-reproducible. We pinned the conversion to ImageMagick, installed it on both machines, and committed the converted PGM inputs so teammates use identical demo data.

Profiling permissions. Nsight Compute attached to the benchmark but NVIDIA's `ERR_NVGPUCTRPERM` setting blocked GPU counter collection on the development machine. Rather than present partial profiles, we kept the committed CSV matrix as the only performance evidence and documented the profiling workflow (`scripts/run_profiling.ps1`) so it can run once counter access is enabled in the driver settings.

7. Conclusion and Future Improvements

The project delivers what Submission 1 proposed: a verified sequential baseline, six CUDA implementations spanning the GPU memory hierarchy, and a 1408-row benchmark matrix per GPU in which every row passes correctness at 1e-6 maximum absolute error against the CPU reference.

Three results summarize the study. Algorithmic structure beats memory tuning when it is available: separable convolution reduces 11x11 work by 5.5x per pixel and holds the overall kernel record (744.2x on the GTX 1650). When the filter is not separable, the memory hierarchy decides: constant-memory filter coefficients halve the best direct kernel time at 4096x4096 (432.8x), while shared-memory input tiling alone does not beat the naive kernel on this hardware. And data movement bounds the application-level benefit: the same case that runs 744x faster as a kernel runs 56x faster end-to-end, so transfer overhead, not computation, is the practical ceiling. The RTX 4070 rerun strengthens all three findings on stronger silicon: 3.7x to 5.4x faster raw kernels, 2383 GFLOP/s peak, identical version ranking, and an even more transfer-dominated total time.

Future work follows from the limitations. Pinned host memory, CUDA streams, and overlapped transfers attack the total-time ceiling, which is now the dominant cost on both GPUs. Enabling GPU performance counters would let the committed Nsight workflow explain kernel behavior with occupancy and memory-throughput numbers instead of timing inference. The comparison could grow toward FFT-based and Winograd convolution and a cuDNN reference point, and toward RGB or batched images, where transfer amortization changes the end-to-end picture. An auto-tuner over block shapes is a natural extension of the sweep, since our data shows the best launch configuration shifts with image size, filter size, and GPU generation.

8. References

Allusse, Y., Horain, P., Agarwal, A., & Saipriyadarshan, C. (2008). GpuCV: A GPU-accelerated framework for image processing and computer vision. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 5359 LNCS(PART 2), 430–439.
https://doi.org/10.1007/978-3-540-89646-3_42

- Iandola, F. N., Sheffield, D., Anderson, M. J., Phothilimthana, P. M., & Keutzer, K. (2013). Communication-minimizing 2D convolution in GPU registers. *2013 IEEE International Conference on Image Processing (ICIP)*, 2116–2120.
<https://doi.org/10.1109/ICIP.2013.6738436>
- Jorda, M., Valero-Lara, P., & Pena, A. J. (2019). Performance evaluation of cuDNN convolution algorithms on NVIDIA Volta GPUs. *IEEE Access*, 7, 70461–70473.
<https://doi.org/10.1109/ACCESS.2019.2918851>
- Lu, G., Zhang, W., & Wang, Z. (2020). Optimizing GPU memory transactions for convolution operations. *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, 399–403.
<https://doi.org/10.1109/CLUSTER49012.2020.00050>
- Perrot, G., Domas, S., & Couturier, R. (2016). An optimized GPU-based 2D convolution implementation. *Concurrency and Computation: Practice and Experience*, 28(16), 4291–4304.
<https://doi.org/10.1002/cpe.3752>
- Van Werkhoven, B., Maassen, J., Bal, H. E., & Seinstra, F. J. (2014). Optimizing convolution operations on GPUs using adaptive tiling. *Future Generation Computer Systems*, 30(1), 14–26.
<https://doi.org/10.1016/j.future.2013.09.003>